



TOWARDS AUTOMATED AML IN DEFI: A GRAPH-BASED APPROACH TO REAL-TIME TRANSACTION ANALYSIS

By

YASH NIMBALKAR

SUPERVISED BY

PROFESSOR MIHAI ORDEAN

A project report submitted to
the University of Birmingham
in partial fulfillment for the degree of
MSC. CYBER SECURITY

School of Computer Science
College of Engineering and Physical Sciences
University of Birmingham
September 2025

DISCLAIMER

I certify that this project is my own work. Code development was assisted by Gemini 2.5 Pro and debugging by Claude Sonnet 4. The report structure and initial drafts are mine; final editing was assisted by ChatGPT for clarity and consistency. All outputs have been reviewed and edited to accurately reflect my work.

ACKNOWLEDGMENTS

I want to express my sincere gratitude towards my supervisor, Prof. Mihai Ordean, for his wisdom, advice and support throughout the duration of this master's thesis. He created a warm and welcoming atmosphere in the meetings, which helped me think critically and complete this thesis in a timely and successful manner.

Next, I would like to express special thanks and gratitude to my parents, Vijay Nimbalkar and Shubhangi Nimbalkar, and my family & friends for their constant encouragement, support, and patience. Their faith has been a continuous source of inspiration and motivation for me.

Ultimately, I am grateful for the University of Birmingham's supportive academic environment, which enabled me to conduct this research. The effective completion of this dissertation was only possible through access to various library resources, collaborating with some amazing peers, and a vibrant academic community.

ABSTRACT

There has been explosive growth in the field of decentralised finance (DeFi), and it has created new compliance and security challenges, particularly in detecting illicit fund flows on public blockchains. With Total Value Locked exceeding \$3.8 trillion globally as of August 2025, DeFi protocols such as decentralised exchanges, cross-chain bridges, and mixers remain vulnerable to sophisticated money laundering techniques, interaction with sanctioned entities, and the movement of stolen assets.

This dissertation studies the feasibility of a real-time, risk assessment system that evaluates blockchain transactions before on-chain services process them. The framework leverages Graph Neural Networks (GNNs) to model transaction graphs and classify patterns of activity as either benign or malicious. A labelled dataset was constructed by tracing over 70 major crypto exploits and contrasting their laundering behaviours, such as peeling chains, scattering, and mixer interactions with legitimate user activity. Rule-based pruning strategies and feature engineering were used to capture structural, behavioural, temporal, and gas-related characteristics of wallet interactions.

Four machine learning models were evaluated, which include Random Forest, GraphSAGE (Graph Sample and Aggregation), Graph Isomorphism Network (GIN), and Graph Attention Network (GAT). Results show that GNNs substantially outperform traditional approaches, with the GIN achieving the highest detection performance by assigning greater weight to critical high-risk interactions.

However, production viability analysis reveals that current performance levels fall short of the 99.99% precision threshold required for autonomous transaction blocking in DeFi systems. At Ethereum's daily volume of 1.5 million transactions, the 5% false positives would create unacceptable disruption for legit users. Consequently, this work suggests the exploration of hybrid AI-human feedback loops and post-transaction analysis frameworks as potential solutions that leverage ML predictions while maintaining user experience in future research.

This work contributes a practical foundation for enhanced DeFi compliance while identifying specific precision improvements necessary for fully autonomous deployment, demonstrating both the potential and current limitations of graph-based machine learning in strengthening financial security for decentralised environments.

Contents

	Page
1 Introduction	1
1.1 Background & Motivation	1
1.2 Problem Statement	2
1.3 Research Objectives & Contributions	2
1.3.1 Research Objectives	2
1.3.2 Key Contributions	3
1.4 Scope & Limitations	3
2 Literature Review	4
2.1 Regulatory and Policy Perspectives	4
2.2 Unsupervised and Anomaly Detection Approaches	4
2.3 Supervised and Active Learning Approaches	5
2.4 Graph-Based Learning Methods	5
2.5 Streaming and Real-Time Detection	5
2.6 Characterisation Studies of Laundering Patterns	5
2.7 AI and Multi-Disciplinary Perspectives	6
2.8 Summary	6
3 Crypto Money Laundering	7
3.1 Core Laundering Strategies	7
3.2 Behavioural Patterns and Technical Characteristics	9
3.3 Detection Challenges and Implications	11
4 Building The System	13
4.1 System Architecture Overview	13
4.2 Dataset Creation & Collection	14
4.2.1 The Challenge of Ground Truth Data	14
4.2.2 Initial Approach and Discovered Bias	14
4.2.3 Refined Data Collection Methodology	15
4.3 Data Collection Implementation	16
4.3.1 Data Source Selection	16
4.3.2 Token Selection Strategy	16
4.3.3 Risk Categories and Address Classification	16

4.4	Transaction Tracing Algorithm	17
4.4.1	Breadth-First Search Implementation	17
4.4.2	Tracing Rules and Logic	17
4.4.3	Algorithmic Parameters	18
4.5	Feature Engineering	18
4.5.1	Node Features	18
4.5.2	Edge Features	19
4.5.3	Feature Weighting Strategy	19
4.6	Data Preprocessing	20
4.6.1	Graph Construction Process	20
4.6.2	Output Format and Storage	21
4.7	Model Implementations	21
4.7.1	Random Forest Implementation	21
4.7.2	GraphSAGE Implementation	22
4.7.3	Graph Attention Network Implementation	23
4.7.4	Graph Isomorphism Network Implementation	24
4.8	Model Deployment Architecture	25
4.9	Implementation Challenges	26
5	Evaluation Results and Analysis	27
5.1	Overview	27
5.2	Evaluation Setup	27
5.2.1	Dataset Configuration	27
5.2.2	Evaluation Metrics Selection	27
5.3	RQ1: Model Effectiveness Analysis	28
5.3.1	Key Findings	29
5.3.2	Feature Behaviour Analysis	29
5.4	RQ2: Production Viability Assessment	30
5.5	RQ3: Real-Time Performance Evaluation	30
6	Conclusion and Future Work	32
6.1	Research Summary	32
6.2	Future Work	33
A	Appendix	34
A.1	Project Repository	34
A.2	Risky Address Data Sources	34
A.3	Ethereum Transaction Fields	34
A.4	Dataset Statistics	34
A.5	Supplementary Figures	35
	References	39

List of Figures

3.1	Example of Peeling Chain Pattern	8
3.2	Example of Chain Hopping Pattern	9
3.3	Using Mixer for obfuscation	9
3.4	Example of a Dusting Attack	10
4.1	Proposed System Architecture	13
4.2	Forward Tracing showing Dispersion Pattern of Funds from a Compromised Starting Address to Multiple Wallets	14
4.3	Backward Tracing depicting How Funds Consolidate from Multiple Sources to a Single Address	15
4.4	Diagram of Random Forest showing construction of multiple decision trees from sample data and aggregation via majority vote	22
4.5	Illustration of the GraphSAGE sample-and-aggregate mechanism. Nodes are sampled from the local neighbourhood and their features aggregated to compute a target node’s embedding (<i>Adapted from Hamilton et al., 2017</i>)	23
4.6	Illustration of the Graph Attention Network architecture. Left: The attention mechanism used in GAT. Right: Multi-head attention applied by node 1 on its neighbourhood, with features aggregated from each head. (<i>Adapted from Veličković et al., 2017</i>)	23
4.7	Architecture of a Graph Isomorphism Network (GIN). The model passes graph-structured input through multiple GIN layers with pooling to form graph-level representations, which are concatenated and fed into a classifier.	24
4.8	Model Predictions for a Sample Trace	25
A.1	Distribution of risky address categories compiled for this study.	36
A.2	Example of a transaction flagged as risky with interaction markers for a known attacker.	37
A.3	Detailed performance metrics for different model architectures.	37
A.4	Time delta distribution comparing malicious (red) vs normal (blue) transactions, with lower deltas indicating faster fund movement	38
A.5	Average preprocessing and model inference times across addresses.	38

List of Tables

4.1	Extracted Node Features	19
4.2	Extracted Edge Features	20
5.1	Model Performance Metrics	28
5.2	Average Pipeline Processing Times (in seconds)	30
A.1	Ethereum Transaction Trace Fields and Their Descriptions	35
A.2	Transaction Statistics Across Malicious and Normal Datasets	36

Acronyms

AML Anti-Money Laundering. 1, 4, 6, 11, 32

AMM Automated Market Maker. 1, 11

CEX Centralised Exchange. 2, 16, 17, 26

DEX Decentralised Exchange. 1, 2, 11, 13, 15–17, 26

DeFi Decentralised Finance. 1, 2, 7, 9, 11, 16, 30, 32

GAT Graph Attention Network. 3, 23, 24, 28, 29

GIN Graph Isomorphism Network. 3, 24, 28–30, 32

GNN Graph Neural Network. 2, 3, 21, 22, 24, 29, 32

GraphSAGE Graph Sample and Aggregate. 3, 22, 28, 29

KYC Know Your Customer. 1, 4, 11, 26

RF Random Forest. 3, 21, 29, 32

TVL Total Value Locked. 1

Chapter One

Introduction

1.1 Background & Motivation

Over the past decade, Decentralised Finance (*DeFi*) has emerged as one of the most transformative applications of blockchain technology. By replacing centralised intermediaries with autonomous smart contracts, *DeFi* protocols offer permissionless access, transparency, and composability. These principles have enabled services such as Decentralised Exchange (*DEX*), Automated Market Maker (*AMM*), lending platforms, and cross-chain bridges. As of August 2025, the Total Value Locked (*TVL*) across various *DeFi* protocols exceeds \$3.8 trillion globally, underscoring both its rapid adoption and its significance within the global financial landscape[1].

Yet, the very features that make *DeFi* compelling, like pseudonymity, immutability, and decentralisation, also introduce profound challenges. Malicious actors exploit these characteristics to launder stolen funds, evade sanctions, and obfuscate illicit financial flows with techniques such as peeling chains, scattering, and mixer interactions. High-profile incidents, such as the *Lubian exploit(2020)*[2] and the *Bybit exploit(2025)*[3], have highlighted both the scale of financial crime in *DeFi* and the difficulty of addressing it. The paradox of blockchain systems lies in their dual nature: while blockchains provide immutable and publicly verifiable records of all activity, the absence of identity binding and the ability to automate and create unlimited wallet addresses make real-time risk assessment highly complex.

Traditional compliance frameworks such as Know Your Customer (*KYC*) and Anti-Money Laundering (*AML*) controls are ill-suited to the decentralised context. Existing blockchain analytics tools are typically forensic, reactive and reliant on manual investigation, leaving a critical gap for solutions that can operate in real time at protocol speed. As regulatory bodies intensify their scrutiny of *DeFi*, the need for scalable, automated, and accurate risk assessment mechanisms has become urgent.

This tension between openness and compliance motivates the development of a high-fidelity, automated risk assessment layer capable of screening incoming blockchain transactions before they are processed without undermining decentralisation. Such a system would not only strengthen security for *DeFi* protocols but also provide a foundation for sustainable growth in the face of evolving regulatory and security challenges.

1.2 Problem Statement

DeFi systems such as Centralised Exchange (*CEX*), *DEX*s, cross-chain bridges, and mixers face a critical challenge: fulfilling compliance obligations and securing their platforms against illicit funds, while maintaining permissionless access. The pseudonymous nature of blockchain transactions, coupled with the ability to generate a high number of wallet addresses, makes it difficult to assess the risk of incoming transactions in real time. This has allowed attackers to launder stolen assets and sanctioned entities to move funds using various advanced techniques.

Current blockchain risk assessment approaches suffer from several limitations:

- **Limited contextual analysis:** Rule-based systems and blacklists fail to capture complex behavioural and structural laundering patterns, often detecting threats only after significant damage
- **Scalability and generalisation :** Many solutions are protocol-specific or require extensive manual tuning, resulting in fragmented and inconsistent security standards across the ecosystem
- **False positive management:** Conservative models generate excessive false positives, degrading user experience, while permissive models miss sophisticated illicit flows

These shortcomings expose protocols to financial loss, reputational damage, and regulatory penalties, especially as oversight from bodies such as the *Financial Action Task Force* (FATF) and the *U.S. Office of Foreign Assets Control* (OFAC) intensifies.

1.3 Research Objectives & Contributions

This dissertation investigates the feasibility of deploying Graph Neural Network (*GNN*) for real-time, on-chain risk assessment of cryptocurrency transactions. The primary objective is to design and evaluate a generalised machine learning framework that can analyse transaction graph structures and behavioural patterns in order to provide probabilistic risk scores for wallets trying to interact with on-chain entities.

The research is structured around three guiding questions that together frame the scope of the study:

- **RQ1: Model Effectiveness:** Can *GNN* approaches outperform traditional machine learning methods for cryptocurrency money laundering detection?
- **RQ2: Production Viability:** Are the achieved performance levels sufficient for practical deployment in real-world DeFi protocols, given current transaction volumes and false positive tolerance requirements?
- **RQ3: Real-Time Performance:** Can the system meet the computational time requirements necessary for real-time transaction screening in production environments?

1.3.1 Research Objectives

In order to address these research questions, the work sets out to:

- Develop and benchmark a labelled dataset of Ethereum transaction graphs that capture both malicious laundering activity and benign user flows, and use this to compare traditional machine learning methods like Random Forest (*RF*) against *GNN* architectures such as Graph Isomorphism Network (*GIN*), Graph Attention Network (*GAT*), and Graph Sample and Aggregate (*GraphSAGE*).
- Evaluate model precision, recall, and false positive rates at scale, and analyse their practical viability for real-world deployment in DeFi protocols where even small misclassification rates can disrupt legitimate user activity.
- Assess preprocessing and inference times across models, and evaluate whether the end-to-end pipeline can support real-time decision-making in production environments.

1.3.2 Key Contributions

The key contributions of this dissertation include:

- **Dataset and Pipeline Creation:** A labelled Ethereum dataset of malicious and normal transaction graphs was constructed, alongside a data preprocessing pipeline that transforms raw blockchain data into *GNN*-compatible formats while retaining temporal and structural transaction information.
- **Comparative Model Evaluation:** Through systematic benchmarking, this study demonstrates that *GNN* models consistently outperform traditional methods such as *RF* in terms of precision, recall, and overall robustness for money laundering detection.
- **Practical Deployment Insights:** By analysing production viability and computational efficiency, the study highlights where current *GNN*-based solutions are already production-ready (*alert generation, risk scoring*) and where further improvements are necessary (*fully autonomous blocking in DeFi systems*).

1.4 Scope & Limitations

This research focuses specifically on Ethereum blockchain transactions and some ERC-20 tokens due to its mature ecosystem, extensive transaction history, and well-documented attack incidents. While the methodology is designed to be generalisable to other blockchain networks, the current implementation and evaluation are conducted exclusively within the Ethereum ecosystem.

The study concentrates on detecting patterns associated with cryptocurrency theft, money laundering, and interaction with sanctioned entities, rather than other forms of blockchain-based illicit activity such as fraud, market manipulation, or regulatory violations. The temporal scope of the analysis covers major cryptocurrency incidents and normal transaction patterns, providing a substantial dataset for model training and evaluation.

Chapter Two

Literature Review

The detection of money laundering in cryptocurrency ecosystems has been thoroughly examined from multiple perspectives, ranging from regulatory frameworks to advanced machine learning and graph-based models. The following review organises prior research into distinct methodological and thematic categories, highlighting both the progress made and the limitations that motivate this study.

2.1 Regulatory and Policy Perspectives

Several works address the regulatory dimension of *AML* in the context of cryptocurrencies. Browne [4] argued for an “enforcement-first” paradigm by extending *KYC* protocols beyond exchanges to mobile phone and email providers, thus aiming to close gaps in user identification. Carucci [5] further analysed how U.S. and EU *AML* frameworks adapt to digital assets, evaluating their effectiveness in practice. Such regulatory studies emphasise that technological tools alone are insufficient without supportive legal and compliance infrastructures.

2.2 Unsupervised and Anomaly Detection Approaches

Unsupervised methods have been widely applied under the assumption that illicit transactions manifest as statistical outliers. Paula et al. [6] demonstrated the effectiveness of deep learning anomaly detection models for fraud, including money laundering in Brazilian exports. Stefánsson et al. [7] and Andersson and Olsson [8] investigated anomalous Bitcoin addresses and developed configurable tracing tools, observing the recurring use of “two-transaction addresses” to obfuscate flows. Similarly, Hu et al. [9] and Jourdan et al. [10] characterised illicit Bitcoin activity by identifying discriminative features in transaction flows and entities. However, Lorenz et al. [11] reported that state-of-the-art anomaly detection algorithms (*e.g.*, *Isolation Forest*, *PCA*, *LOF*) perform poorly on real transaction data, indicating that illicit activity often mimics legitimate behaviour rather than appearing as clear outliers.

2.3 Supervised and Active Learning Approaches

Supervised models have shown strong potential when sufficient labelled data is available. The release of the Elliptic dataset [12] has driven significant research. To address the challenge of scarce labels, Lorenz et al. [11] proposed an active learning approach that achieved performance comparable to that of supervised methods while using only 5% of the labels. Deng et al. [13] demonstrated similar effectiveness with reduced label requirements in financial fraud detection. These contributions highlight the practicality of label-efficient strategies for real-world deployment.

2.4 Graph-Based Learning Methods

The structural nature of blockchain transactions naturally lends itself to graph modelling. Multiple studies have advanced this paradigm:

- **Subgraph-Based Models:** Ouyang et al. [14] proposed Bit-CHetG, a contrastive learning approach leveraging heterogeneous subgraphs to identify laundering groups, outperforming baselines in Micro-F1 score.
- **Graph Neural Networks:** Weber et al. [15] introduced Graph Convolutional Networks (GCNs) and EvolveGCN to AML, pioneering the use of GNNs in this domain. Adloori et al. [16] later developed GAT-ResNet, a Graph Attention Network with residual connections, achieving higher scalability and accuracy.
- **Transaction Flow Models:** Li et al. [17] presented FlowScope, a multipartite graph method modelling entire transaction flows to improve detection granularity.

These studies collectively highlight the promise of graph-based approaches in capturing both micro (node-level) and meso (subgraph-level) laundering patterns.

2.5 Streaming and Real-Time Detection

Sun et al. [18] introduced MonLAD and its sliding-window variant MonLAD-W, scalable sketching algorithms designed for streaming transaction data. By tracking account residuals and statistical features, they enabled robust real-time detection. Their A3S algorithm further enhanced anomaly identification under noisy conditions. This body of work underscores the shift from static analysis toward streaming-compatible AML solutions.

2.6 Characterisation Studies of Laundering Patterns

Several works have contributed empirical insights into laundering behaviour. Andersson and Olsson [8] identified illicit flows involving Darknet markets, ransomware, and tumblers, emphasising patterns such as reliance on intermediaries and two-transaction addresses. Dan Lin et al. [19] investigated Ethereum-based laundering strategies in Web3 contexts, highlighting emerging techniques such as counterfeit tokens and speculator masquerading.

2.7 AI and Multi-Disciplinary Perspectives

Basu and Tetteh [20] evaluated the strengths and weaknesses of Artificial Intelligence in *AML*, advocating for hybrid approaches that combine machine learning, natural language processing, and network analysis within regulatory frameworks. This reflects a growing consensus that technical, regulatory, and operational strategies must evolve in tandem to combat increasingly sophisticated laundering schemes.

2.8 Summary

The literature reveals that while unsupervised anomaly detection has produced mixed results, supervised and graph-based learning—especially with active learning and subgraph modelling—show strong promise. Streaming detection and explainability are emerging frontiers, bridging the gap between technical performance and operational viability. These gaps in current research provide the foundation for the objectives of this dissertation.

Chapter Three

Crypto Money Laundering

By leveraging the pseudonymity and composability inherent to *DeFi* platforms, malicious actors have developed increasingly sophisticated techniques to launder money. These methods represent a natural evolution of traditional laundering practices, now adapted to fit the decentralised, often opaque, landscape of blockchain technology. In exploiting the permissionless nature of *DeFi* and the current gaps in regulatory oversight, criminals are able to bypass conventional monitoring tools and reduce the traceability of illicit funds.

3.1 Core Laundering Strategies

- **Peeling Chains:** This common method involves distributing large sums of cryptocurrency through a sequence of smaller transactions across multiple wallet addresses. Here, funds are broken down incrementally, with small portions moved at each step, creating a tangled web of transactions. This fragmentation mimics legitimate activity and makes forensic analysis far more difficult, particularly as the number of addresses and transfers grows. As you can see in the Figure. 3.1 the attackers of the *Ronin Bridge(2022)*[21] used exactly this similar method to transfer the funds in various wallets multiple times, and then eventually all the money was sent to Tornado.Cash a mixing service.
- **Chain Hopping:** To complicate tracking efforts, launderers frequently move assets across multiple chains using decentralised exchanges and cross-chain bridges. This “chain hopping” technique takes advantage of the fact that the wallet addresses change when moving funds from one blockchain to another. By cycling assets between platforms such as Ethereum, Binance Smart Chain, and Avalanche, criminals can introduce jurisdictional and technical barriers that frustrate investigators. The Figure. 3.2 is derived from the *Bybit Exploit(2025)*[3], and we can observe that the Lazarus group used services like THORChain for converting the funds into Bitcoin. Although, the attackers also tried to move the funds via ChainFlip, but the team at Chainflip promptly upgraded its protocol to block transactions linked to the Bybit hack and publicly stated it would not facilitate such illicit transfers.
- **Mixing Services:** Mixers or tumblers are services that obfuscate the link between the sender and receiver by pooling funds from various users and redistributing them

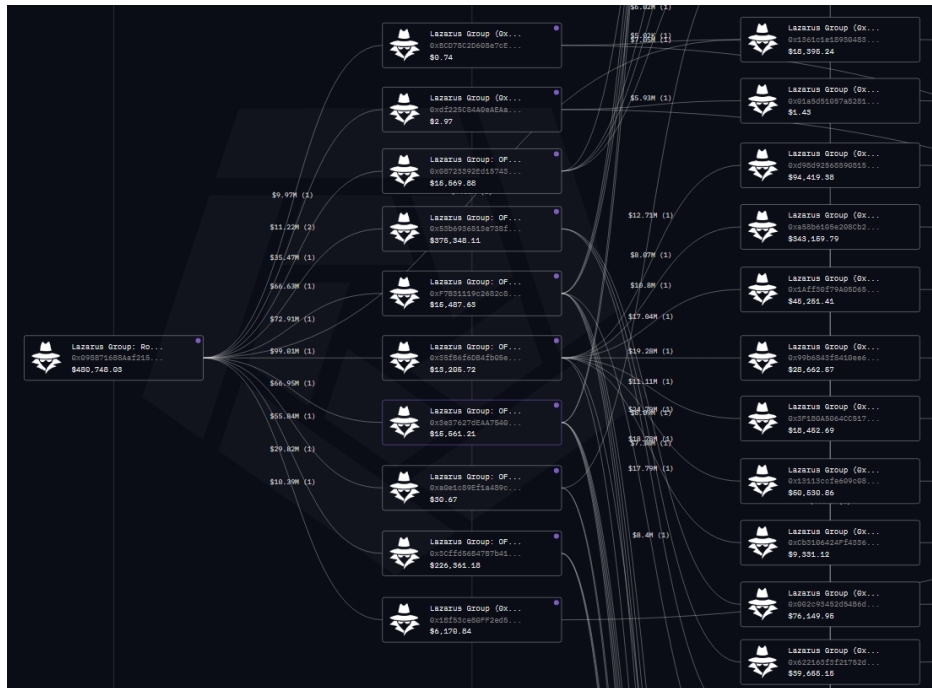


Figure 3.1: Example of Peeling Chain Pattern

in randomised amounts. Tools like Tornado Cash, Wasabi Wallet, and other privacy-enhancing protocols use advanced cryptographic techniques such as zero-knowledge proofs and ring signatures to guarantee unlinkability. And newer versions of mixers have added features like delayed withdrawals and variable transaction sizes to further obscure fund flows. In the case of *Fei Protocol exploit(2022)*[22], by using a combination of flash loans and re-entrancy, the attackers were able to steal \$80M. The attackers then used Tornado.Cash mixer to vanish the funds (Ref. Figure 3.3).

- **Dusting Attacks:** Originally this technique was designed to de-anonymise wallet owners, but dusting attacks are now also being adapted for laundering. In this method, criminals send tiny amounts of cryptocurrency to many wallets. These transactions can later be used to create plausible justifications for larger transfers or to contaminate otherwise clean addresses with traces of illicit funds. In the figure 3.4, we observe multiple small-value transactions from various addresses—including those flagged for phishing—converging into a single recipient address, illustrating a typical dusting-like pattern possibly used to obscure fund origins.
- **Privacy Coins:** Coins like Monero or Zcash are specifically designed to obscure transaction data through built-in privacy features, such as stealth addresses, confidential transactions, and ring signatures. Once assets are converted into these coins, the transactional trail effectively vanishes from public blockchain records. Launderers may then hold, fragment, or further circulate these assets before converting them back into more widely used tokens.
- **NFT-Based Laundering:** Non-Fungible Tokens (NFTs) have emerged as another

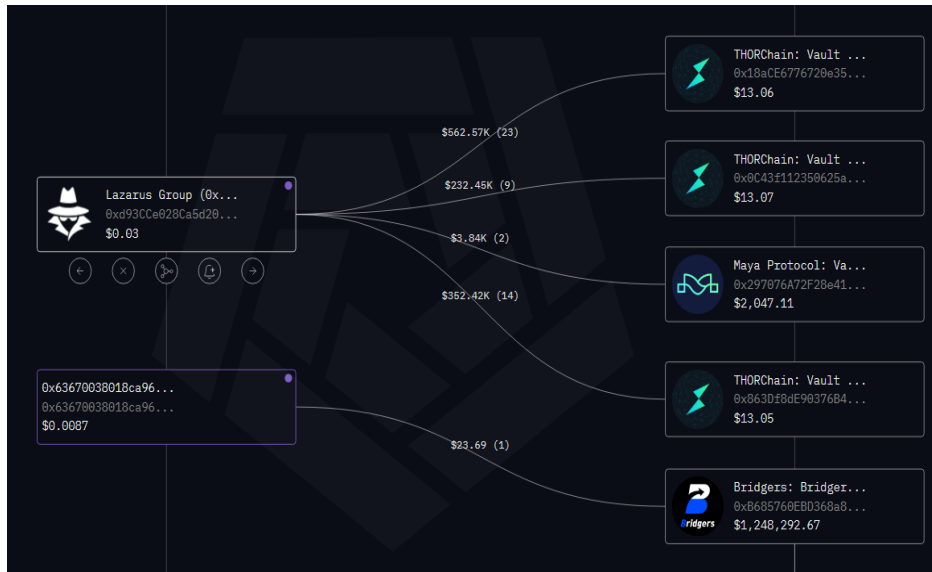


Figure 3.2: Example of Chain Hopping Pattern

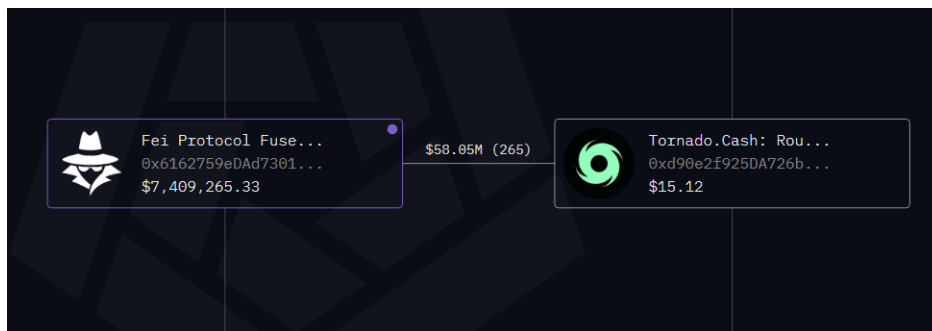


Figure 3.3: Using Mixer for obfuscation

tool for money laundering due to their non-standardised pricing and subjective valuations. Criminals can mint NFTs at low cost and then “sell” them to themselves or collaborators at artificially inflated prices, thereby justifying the movement of large sums under the guise of legitimate asset sales.

- **Flash Loan Laundering:** Flash loans are instant and uncollateralized loans that must be repaid within a single transaction. These tools allow attackers to access substantial liquidity without needing upfront capital. Through complex smart contract interactions, they can rapidly cycle funds through various DeFi protocols in a way that’s extremely difficult to trace and analyse.

3.2 Behavioural Patterns and Technical Characteristics

- **High-Velocity Transactions:** A well-known pattern of laundering operations in *DeFi* is the unusually high frequency of transactions in short time spans. Dozens or

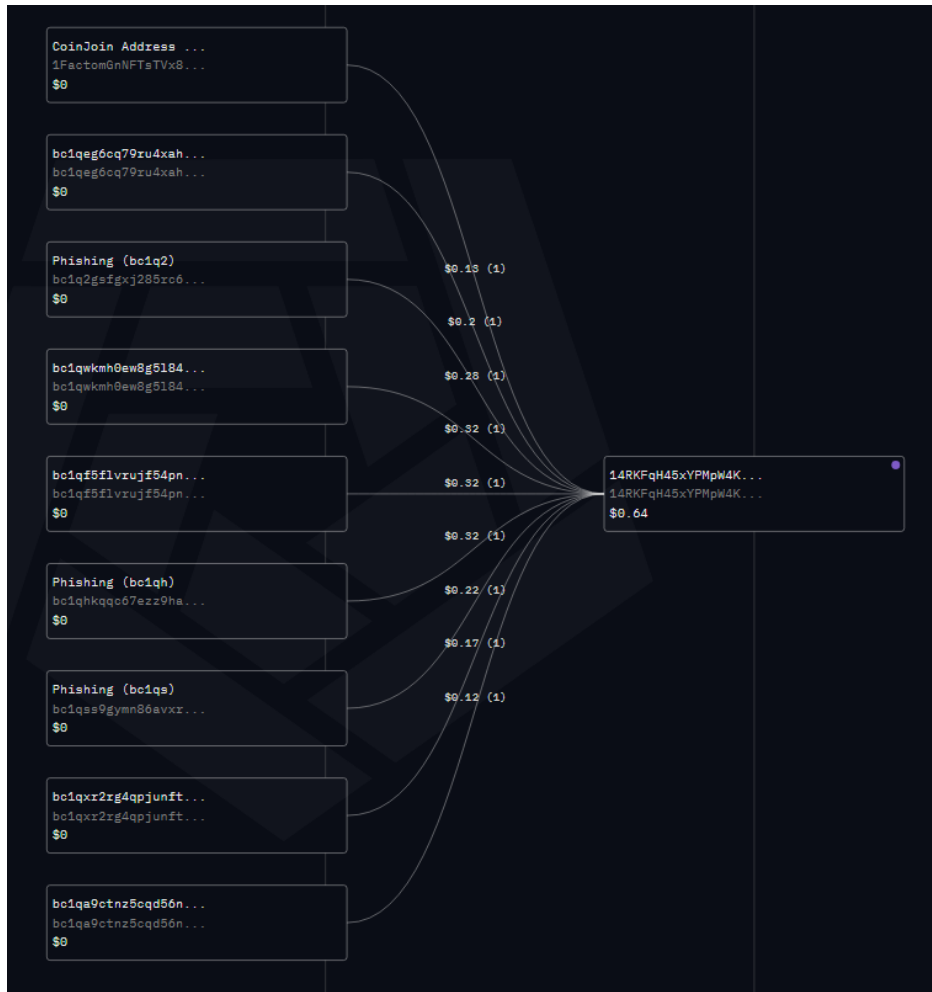


Figure 3.4: Example of a Dusting Attack

even hundreds of transactions might occur within minutes. This high throughput is designed to overwhelm monitoring systems and obscure illicit patterns through sheer transaction volume.

- **Elevated Gas Expenditure:** Suspicious gas fee patterns often accompany laundering activities. These include:
 - *High gas prices*, to ensure quick confirmations for time-sensitive transactions
 - *Inefficient gas use*, as complex smart contracts often consume more gas than typical user behaviour
- **Multi-Account Usage:** Organized laundering schemes often involve the use of multiple wallets in parallel:
 - *Parallel processing*, where many wallets move funds simultaneously to increase throughput and reduce detection risk
 - *Hierarchical structuring*, where funds pass through several layers of intermediary addresses

- *Address rotation*, a tactic used to evade long-term behavioural profiling by blockchain analysts
- **Smart Contract Exploitation:** Launderers take full advantage of *DeFi*'s programmable nature:
 - *DeFi protocol hopping*, where assets are rapidly moved across lending, trading, and staking protocols to create complex trading patterns
 - *AMM manipulation*, using price slippage and token swaps to disguise the trail of funds
 - *Yield farming abuse*, which enables criminals to earn returns while simultaneously using protocol participation as a cover for laundering

3.3 Detection Challenges and Implications

Despite advancements in real-time *AML* technology for cryptocurrencies, including the deployment of clustering heuristics, machine learning, and blockchain analytics by leading firms, money laundering remains a persistent issue. Several fundamental problems explain why even the most powerful systems struggle to stop laundering instantly:

- **Pseudonymous Nature:** While all blockchain transactions are public, wallet owners are rarely associated with verifiable real-world identities. Privacy-preserving protocols, pseudonymous addresses, and *DeFi* services allow criminals to move funds without direct attribution, making it challenging to identify suspicious actors, even when risky flows are flagged.
- **Rapid Evolution of Techniques:** Money launderers evolve faster than detection systems. Methods such as peeling chains, mixers, nested services, privacy coins, and cross-chain bridges continue to get enhanced to beat detection models. Adversaries create new evasion methods as aml systems adapt, creating a cat-and-mouse dynamic.
- **Decentralized and Global Structure:** Standard safeguards, such as *KYC* checks and fund freezes, are ineffective for many laundering methods that use *DEX* and *DeFi* protocols. Criminals may also use enforcement loopholes created by regulatory differences between states when transferring assets abroad.
- **Data and Model Limitations:** Effective AI-based *AML* requires robust, labelled datasets reflecting real laundering cases. However, such publicly available data is scarce, incomplete, or intentionally obfuscated by criminals. This scarcity hampers model reliability and makes it difficult to generalize across chains, tokens, and evolving laundering strategies.
- **Adversarial Attacks:** Criminals actively test detection algorithms for flaws. By creating transaction patterns that imitate benign behavior, they can exploit model blind spots and misclassifications, lowering the effectiveness of existing AI-based *AML* solutions.
- **Cost and Complexity:** Real-time *AML* systems require significant computational and financial resources to run. Even when suspect transfers are detected, decentralized

peer-to-peer networks avoid direct intervention: by the time a risk is identified, funds may have already moved to another ecosystem or asset class.

- **Enforcement Gaps:** Although some centralized exchanges have strong compliance processes, others are susceptible to poor or inconsistent regulatory oversight. These safe havens restrict authorities' ability to impose sanctions or freeze illegal assets, creating opportunities that sophisticated launderers exploit.

In summary, the persistence of cryptocurrency money laundering is due to a combination of pseudonymity, rapid adversarial adaptation, decentralized architecture, limited training data, adversarial exploitation of AI models, and unequal worldwide legislation, rather than a lack of monitoring technologies. While existing analytics solutions provide significant retrospective insights, the creation of generic, adaptive, and predictive risk assessment systems remains an open research topic, which this dissertation tries to address.

Chapter Four

Building The System

4.1 System Architecture Overview

The proposed solution operates as an automated screening system that analyses Ethereum transactions before they are approved on the blockchain. When a wallet initiates a transaction with a partner service, such as a *DEX*, cross-chain bridge, or other DeFi protocol, the system performs a backwards trace of the incoming funds to analyse transaction patterns and detect potential money laundering activities.

The core workflow operates as follows: upon transaction initiation, the system conducts a comprehensive backwards trace from the source wallet, collecting and analysing historical transaction data for all the incoming wallets. This data undergoes preprocessing to extract relevant features, which are then fed into machine learning models trained to distinguish between legitimate and suspicious transaction patterns. If malicious activity is detected, the system flags the transaction and, where possible, enables the on-chain service to stop the transaction or freeze the associated assets. This entire process must be completed in real-time before the transaction receives blockchain approval.

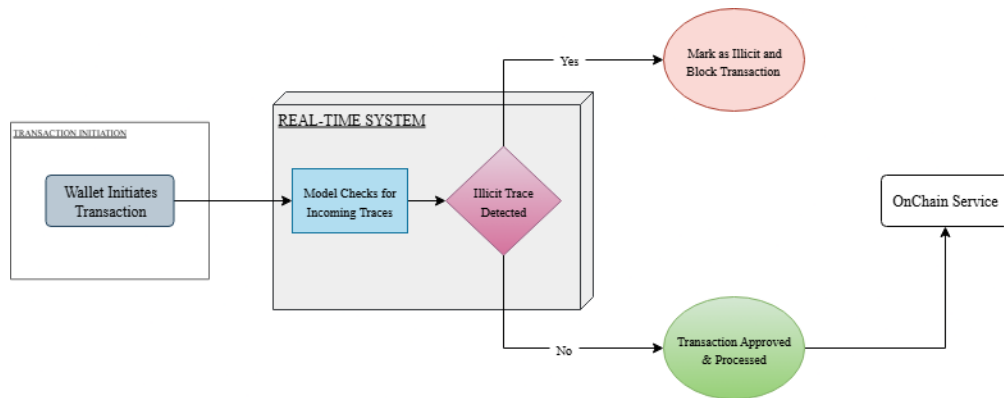


Figure 4.1: Proposed System Architecture

4.2 Dataset Creation & Collection

4.2.1 The Challenge of Ground Truth Data

A significant challenge in developing this system was the absence of publicly available datasets for blockchain transactions labelled as malicious or benign. The only existing dataset, which was provided by Elliptic, does contain over 200,000 bitcoin transactions but only until 2019 [12, 23]. Thus, it was unsuitable for current Ethereum-based analysis. To address this, a custom dataset was created using Ethereum mainnet transaction data collected via Etherscan’s public APIs, accurately capturing both illicit and licit transaction patterns in the Ethereum ecosystem.

4.2.2 Initial Approach and Discovered Bias

Initially, two distinct tracing methodologies were adopted for data collection. Forward tracing was applied to known attacker addresses to capture patterns of outgoing fund dispersion, generating graphs that illustrated how illicit funds were distributed from attacker-controlled wallets (see Fig.4.2). In contrast, backwards tracing was applied to benign addresses to cap-

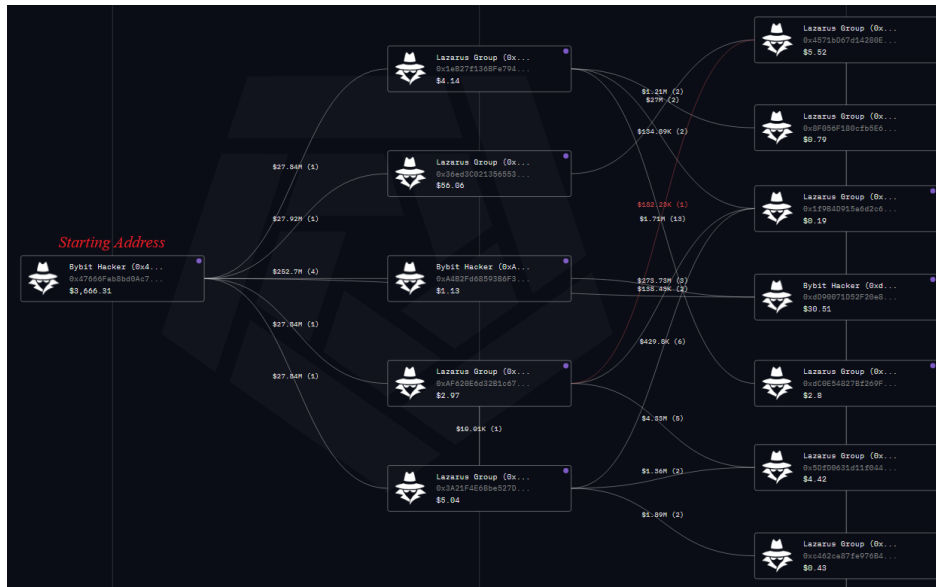


Figure 4.2: Forward Tracing showing Dispersion Pattern of Funds from a Compromised Starting Address to Multiple Wallets

ture patterns of incoming fund consolidation, producing graphs that reflected how legitimate funds accumulate in user wallets (see Fig.4.3).

However, during model training and validation, a critical bias was observed: the models were learning to identify transactions based on the *direction of tracing* rather than underlying behavioural characteristics. Since the proposed system is based purely on backward tracing, such models would eventually classify all addresses as benign. As a result, they

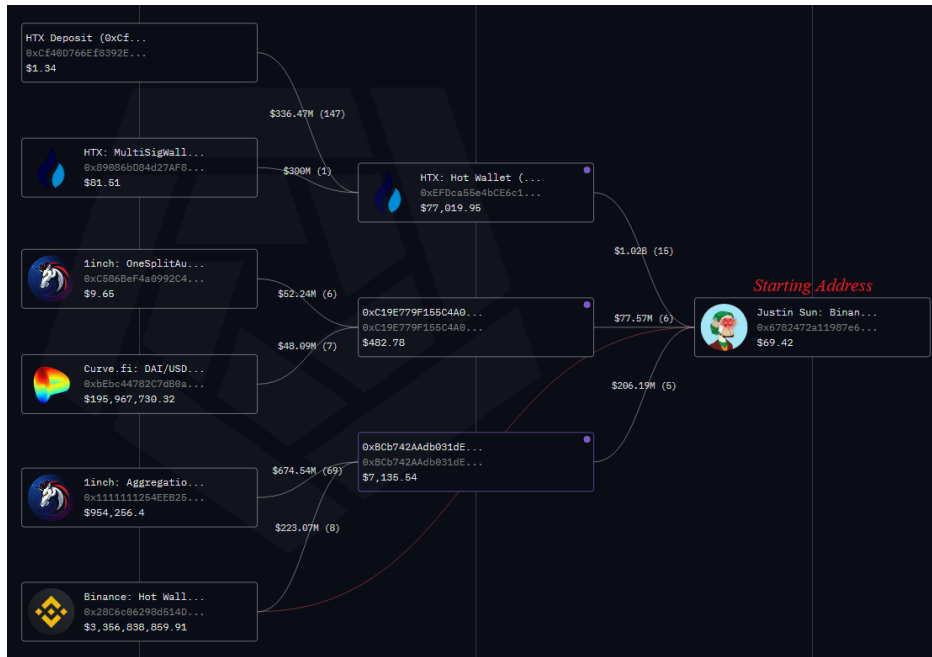


Figure 4.3: Backward Tracing depicting How Funds Consolidate from Multiple Sources to a Single Address

would consistently misclassify transactions in real-world deployment, severely limiting their practical effectiveness.

4.2.3 Refined Data Collection Methodology

To mitigate this bias, the data collection strategy was updated as follows:

- **Malicious Data Collection**

- *Source Identification*: Started from known attacker addresses which were involved in major cryptocurrency hacks
- *Forward Tracing Phase*: Manually followed the the stolen funds until they reached terminal points such as mixers, bridges, exchanges, or *DEX* platforms
- *Backward Tracing Phase*: From these addresses which were interacting with the terminal points, executed the backward tracing to capture the money laundering patterns that the real-time system would encounter

- **Benign Data Collection**

- *Trusted Source Selection*: Identification of verified benign addresses including well-known individuals, reputable exchanges, miners, and legitimate users
- *Backward Tracing*: Applied the same backward tracing algorithm to ensure methodological consistency with the malicious dataset and aligned with the real-time system operation

This refined approach ensured that both malicious and benign samples were collected using the same tracing methodology, thus eliminating directional bias while preserving the

authentic behavioural patterns that distinguish licit and illicit fund flows.

4.3 Data Collection Implementation

4.3.1 Data Source Selection

The data collection process used several Etherscan API calls to acquire comprehensive Ethereum mainnet transaction information, including:

1. *Normal Transactions*: ETH transfers between addresses
2. *Internal Transactions*: Smart contract-generated transfers
3. *Token Transfers*: ERC-20 token movements
4. *Wallet Funding Source*: Historical funding data for address age analysis

Table A.1 presents the data fields used in this study. Most fields are directly obtained from Ethereum transaction records, while a few (e.g., `transaction_type`, `is_risky_tx`, `risk_type_tx`, `fundingAddress`, and `fundingTxn`) were manually added during the preprocessing and annotation stages to support risk analysis.

4.3.2 Token Selection Strategy

The token analysis focused on widely adopted ERC-20 tokens, specifically targeting seven major tokens that represent significant value flows in the Ethereum ecosystem:

- USDT
- USDC
- Liquid Staked Ether (stETH)
- Wrapped liquid staked Ether 2.0
- Wrapped Ether (WETH)
- Wrapped BTC
- DAI Stablecoin

This selection was based on market capitalisation, trading volume, and prevalence in both legitimate and illicit activities.

4.3.3 Risk Categories and Address Classification

A comprehensive risk classification system was developed using the `riskyAddresses.json` file (see Fig.A.1), which contained 9,849 categorised addresses across six categories:

- *CEX*: 4,448 addresses representing major centralized trading platforms
- *DEX*: 149 addresses covering prominent *DeFi* trading protocols
- *Bridges*: 255 addresses facilitating inter-blockchain asset transfers
- *Mixers*: 6 addresses belonging to known mixing services
- *OFAC Sanctioned Addresses*: 46 addresses officially sanctioned by regulatory authorities

- *Known Attackers*: 4,945 addresses associated with various cryptocurrency incidents

This address list was gathered from multiple public sources (see Appendix A.2), combined with manual enumeration and cross-verification. This categorisation serves two main purposes: to identify tracing termination points and to provide risk context for feature engineering.

This list was subsequently utilised during transaction tracing to determine whether nodes or wallets interacted with any listed addresses. Two independent labels, `is_risky_tx` and `risk_type_tx`, were then appended to every traced transaction. The former provided a binary indication of risk, flagging transactions as `true` when they involved categories such as `mixers`, `sanctionedAddresses`, or `knownAttackers`. The latter specified the exact category of risk. If the interaction only involved minor categories that weren't considered risky, only the second label was added. (See Fig.A.2).

4.4 Transaction Tracing Algorithm

4.4.1 Breadth-First Search Implementation

The system uses a Breadth-First Search (*BFS*) approach to reconstruct the transaction histories. BFS was chosen because of its computational effectiveness and ability to examine transaction graphs systematically. The algorithm ensures a balanced search of both breadth and depth by processing all transactions at depth `n` before going to depth `n+1`. This structure allows for the identification of fund flow patterns at every level of the transaction hierarchy.

4.4.2 Tracing Rules and Logic

The tracing process is controlled by a set of rules that specify when to continue traversal and when to stop. These rules are created to capture valuable fund flow behaviour while minimising computational overhead.

Stop Conditions Tracing is terminated under the following circumstances:

- **Exchanges**: When funds reach a *CEX* or *DEX*, tracing stops because these points represent a change of ownership or entry into liquidity pools, breaking direct traceability
- **Bridges**: Tracing stops when assets are moved to cross-chain bridges, pointing to chain-hopping behaviour and breaking the transaction graph
- **Mixers**: Termination happens at mixing services where cryptographic obfuscation prevents further tracking
- **Maximum Depth**: The algorithm stops when the predefined search depth `n` is reached
- **Insufficient Data**: If a transaction lacks sufficient historical inputs, the trace ends at that node

Continue Conditions Tracing continues in most of the conditions, except when it hits the “*Stop conditions*”, else it continues the trace to capture critical behavioural patterns:

- **OFAC-Sanctioned Addresses:** Tracing proceeds through sanctioned wallets to map the complete extent of illicit networks and identify their transaction partners
- **Known Attackers:** Tracing is continued through addresses associated with confirmed malicious actors to capture the full laundering trajectory of stolen funds

4.4.3 Algorithmic Parameters

The tracing algorithm operates with carefully tuned parameters designed to balance comprehensive analysis with computational efficiency:

- **max_depth = 10:** Sets a maximum tracing depth to prevent infinite loops and control computational complexity
- **top_n_incoming = 3:** Considers the three highest-value incoming transactions per address to focus on economically significant flows
- **top_n_outgoing = 2:** Examines the two largest outgoing transactions per address to capture primary fund dispersal patterns
- **value_threshold_ratio = 0.001:** Filters transactions below 0.1% of the largest transaction value to reduce noise from dust transactions

This combination of BFS traversal, rule-based termination logic, and parameter tuning enables the system to balance depth and efficiency. The method ensures that laundering behaviours are adequately captured, while avoiding unnecessary expansion into irrelevant or untraceable transaction paths.

4.5 Feature Engineering

Money laundering activity does exhibit some structural and behavioural patterns in the movement of funds. To train models capable of distinguishing illicit from legitimate activity, it was necessary to extract features that capture these underlying characteristics from raw transaction data. The feature engineering process was used to derive meaningful representations of both addresses and transactions within the transaction graphs. The idea was to capture the *structural role* of an entity within the graph and the *behavioural characteristics* reflected in its transactional activity. Based on this idea, twelve node-level features and eight edge-level features were extracted and grouped into structural, behavioural, temporal, and gas-related categories. Furthermore, a weighting scheme was used to ensure that features with stronger diagnostic relevance exert proportionately greater influence during model training.

4.5.1 Node Features

In our graph data, each wallet is a node. The extracted node features are shown in Table 4.1:

Table 4.1: Extracted Node Features

Category	Feature Name	Description
Structural	depth	Position in the tracing hierarchy, indicating distance from the target address
	is_traced	Binary indicator of whether the address was included in the active tracing path
	in_degree	Number of incoming transaction connections
	out_degree	Number of outgoing transaction connections
Behavioural	total_incoming_value	Log transformed sum of all incoming transaction values
	total_outgoing_value	Log transformed sum of all outgoing transaction values
	in_out_value_ratio	Ratio of incoming to outgoing value, indicating fund accumulation or dispersal patterns
	fund_velocity	Log transformed measure of time between fund receipt and transfer
	wallet_age	Log transformed age of the wallet since its first transaction
	tx_frequency	Number of transactions per unit time
	unique_cp	Count of distinct interacting addresses
Gas Analysis	avg_gas_price	Log transformed average gas price used in transactions, indicating urgency or priority patterns

4.5.2 Edge Features

In our graph data, each transaction is an edge. Table 4.2 shows the extracted edge features:

4.5.3 Feature Weighting Strategy

To emphasise behaviorally significant indicators, a differential weighting scheme is applied:

- Standard Weight (1.0): Basic structural features (*depth*, *is_traced*, *total values*, *degrees*)
- Medium Weight (1.2): Temporal patterns (*wallet_age*, *tx_frequency*)
- Medium-High Weight (1.3): Network behaviour (*unique_counterparties*, *avg_gas_price*)
- High Weight (1.5): Critical behavioural indicators (*in_out_value_ratio*, *fund_velocity*)

This weighting strategy does ensure that the features which are indicative of money laundering behaviour receive appropriate emphasis during model training.

Table 4.2: Extracted Edge Features

Category	Feature Name	Description
Structural	<code>transaction_value</code>	Log transformed transaction amount
	<code>is_internal</code>	Binary indicator for internal (smart contract-generated) transactions
	<code>is_traced_edge</code>	Binary indicator for transactions included in the active tracing path
Risk Labelling	<code>is_risky_tx</code>	Binary indicator for transactions involving high-risk addresses
	<code>risk_type_encoded</code>	Categorical encoding of risk types (<i>"mixers"</i> : 1, <i>"sanctionedAddresses"</i> : 2, <i>"knownAttackers"</i> : 3, <i>"bridges"</i> : 4)
Temporal / Gas	<code>time_delta</code>	Log transformed time difference between consecutive transactions
	<code>gas_price</code>	Log transformed gas price for the specific transaction
	<code>total_gas_cost</code>	Log transformed total gas spent

4.6 Data Preprocessing

4.6.1 Graph Construction Process

Data preprocessing is another critical step that needs to be performed before the data can be passed on to the machine learning models. This step involved transforming the raw transaction data into a structured format which the models could then understand. It consists of four sequential stages.

First, we begin with the fundamental task of node mapping. In this, each unique address within the transaction network receives a distinct numeric identifier. By using a *set* data structure, we ensure that each address occurs only once. This creation of the address-to-index mappings proved to be essential for maintaining data integrity during the preprocessing pipeline.

Next, we proceeded to the most technically demanding aspect of the preprocessing workflow, which was feature engineering. The implementation of node and edge features, as specified in the earlier methodology, required careful attention to the logarithmic transformations, which were necessary to handle the extreme value ranges of the various features in the transaction data. After analysing various transactions, it was evident that raw transaction values could vary in multiple orders of magnitude, from micro-transactions to large transfers from exchanges, etc. Thus, it was necessary to implement normalisation strategies to prevent model bias towards high-value transactions.

The normalisation stage was implemented using scikit-learn's `StandardScaler` library. For this step, separate scaling of node and edge features was performed, and it ensured that each feature type maintained appropriate statistical properties while preserving the graph's

topological characteristics.

The final stage in the processing pipeline involved the conversion to PyTorch Geometric format. In this, PyTorch tensors were created for node features, edge connectivity, and edge attributes, thus creating a standardised graph representation suitable for *GNN* architectures and also enabling efficient batch processing and GPU acceleration for machine learning workflows.

4.6.2 Output Format and Storage

The processed data is then saved as .pt files using PyTorch’s native tensor format. This enables efficient loading during model training and inference. Each output file contains a complete PyTorch Geometric Data object representing a single transaction trace with all associated features and metadata. All the output traces are then stored in an organised and clear directory structure:

- `./Dataset/malicious/`: Contains 100 .pt files representing malicious transaction patterns
- `./Dataset/normal/`: Contains 101 .pt files representing benign transaction patterns

4.7 Model Implementations

Given the graph-structured nature of blockchain transaction data, both traditional machine learning and *GNN* approaches were implemented to assess their usefulness in money laundering detection. Four different models were considered, each having a different approach for handling graph-structured data.

4.7.1 Random Forest Implementation

Random Forest (*RF*) is a machine learning algorithm that uses multiple decision trees to make better predictions [24]. It works by randomly selecting subsets of data and features to build each tree. The randomness helps to reduce overfitting and improve predictive accuracy. The output is then given by combining the results by voting for classification or averaging for regression. (See Fig.4.4)

In this study, the *RF* model was adopted as a baseline for comparison against graph-based approaches. While the *RF* model works well with tabular data, it cannot directly process graph-structured data. So to address this limitation, the collected graph data first had to be transformed such that it could be learned by the *RF* model.

This process of converting the graph structures into fixed-size feature vectors was executed by using a special `featurize_graph()` function. This function extracts fifteen meaningful aggregate statistics from the complex graph data. These included structural descriptors such as node count, edge density, and average depth, alongside economic measures capturing transaction value dynamics. Together, these features provided a compact yet informative

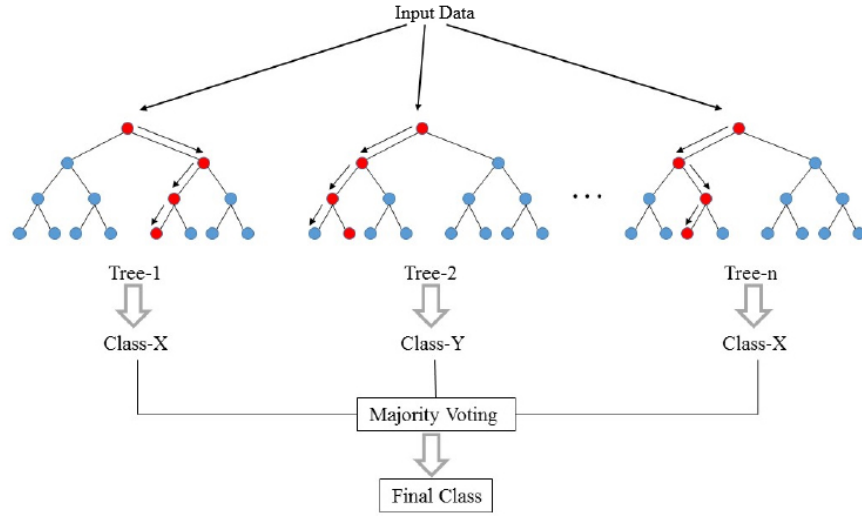


Figure 4.4: Diagram of Random Forest showing construction of multiple decision trees from sample data and aggregation via majority vote

representation of the underlying transaction patterns, enabling the model to capture relevant behavioural characteristics.

For training and testing, the dataset was then split into training and testing sets using an 80:20 split. The `RandomForestClassifier` was configured with 100 estimators, chosen to balance computational efficiency with predictive accuracy, and the class weight parameter was retained at the default *balanced* setting to mitigate any label imbalance.

4.7.2 GraphSAGE Implementation

GraphSAGE is a *GNN* model designed to efficiently generate node embeddings by sampling and aggregating information from a node's local neighbourhood, as illustrated in Figure 4.5. Instead of training on the full graph, GraphSAGE learns an aggregation function that combines the features of a node and its neighbours, enabling inductive learning on previously unseen nodes and subgraphs. This makes it particularly suitable for dynamic networks such as blockchain transaction graphs, where new addresses appear continuously[25].

A three-layer SAGE convolution was selected, with each layer using 64 hidden channels. This configuration was chosen after experimentation to strike a balance between expressive power and computational efficiency. The mean aggregation strategy was adopted as it offered stable learning across the wide range of graph structures observed in cryptocurrency networks, while global pooling operations ensured that both common and extreme behavioural patterns were captured effectively.

Some architectural refinements were also introduced to improve training performance, such as *residual connections* to support gradient flow in deeper networks, and *batch normalisation* was applied to improve convergence stability. A *dropout rate* of 0.2 was selected based on validation experiments as an effective safeguard against overfitting while preserving

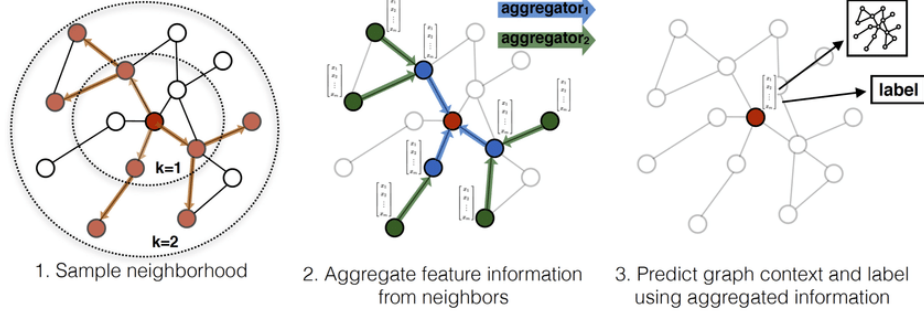


Figure 4.5: Illustration of the GraphSAGE sample-and-aggregate mechanism. Nodes are sampled from the local neighbourhood and their features aggregated to compute a target node’s embedding (*Adapted from Hamilton et al., 2017*)

model capacity. Additionally, *skip connections* were used when layer dimensions matched, to solve the gradient flow issues identified during early training runs.

4.7.3 Graph Attention Network Implementation

The *GAT* introduces an attention mechanism that assigns different weights to neighbouring nodes during message passing, as shown in Figure 4.6. Instead of treating all neighbours equally, *GAT* learns which connections are more important for predicting a node’s behaviour, thereby improving the expressiveness of the model. This selective weighting is especially valuable in blockchain data, where some transactions and wallets may be far more influential in shaping suspicious activity than others[26].

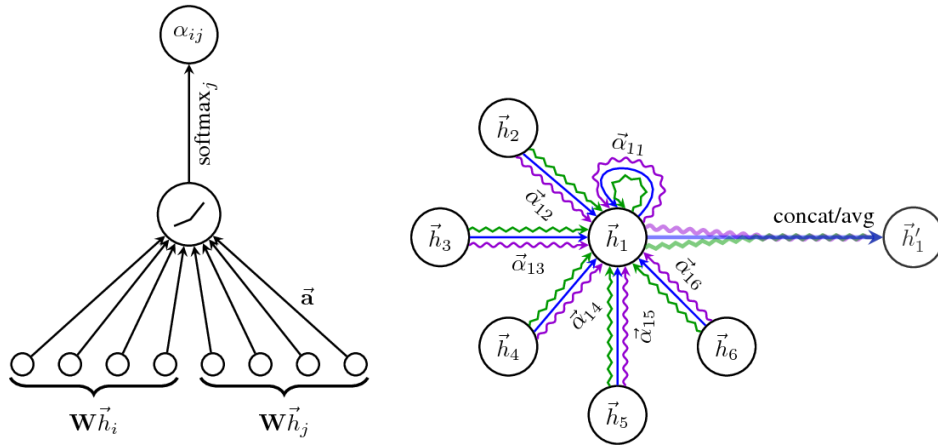


Figure 4.6: Illustration of the Graph Attention Network architecture. **Left:** The attention mechanism used in GAT. **Right:** Multi-head attention applied by node 1 on its neighbourhood, with features aggregated from each head. (*Adapted from Veličković et al., 2017*)

In this study, *GAT* was applied to transaction graphs to capture both structural information and interaction patterns between addresses. Edge features such as transaction value

and gas metrics were encoded into the attention mechanism, allowing the model to learn which transaction flows carry greater significance for risk detection.

For the implementation of *GAT*, a multi-head attention setup with four heads per layer was used, which enabled the model to examine different relationship types simultaneously. Eight-dimensional edge features were directly integrated into the attention weight computation, which enhanced the model’s ability to highlight suspicious transaction behaviours.

The architecture was refined through experimentation with head configurations and hidden channel sizes. Early layers employed 256 channels to retain diverse perspectives, while the output was reduced to 64 channels to concentrate attention in later stages. A concatenation strategy was used in the early layers to preserve multiple perspectives, while averaging in the final layer enabled consensus-based classification decisions.

4.7.4 Graph Isomorphism Network Implementation

The *GIN* is a highly expressive *GNN* architecture. It operates by applying *Multi-Layer Perceptrons* (MLPs) to aggregated neighbour features, as illustrated in Figure 4.7, allowing it to capture subtle structural differences that simpler models may overlook. This expressiveness makes it particularly effective for identifying complex transaction patterns and hidden laundering strategies within blockchain graphs[27].

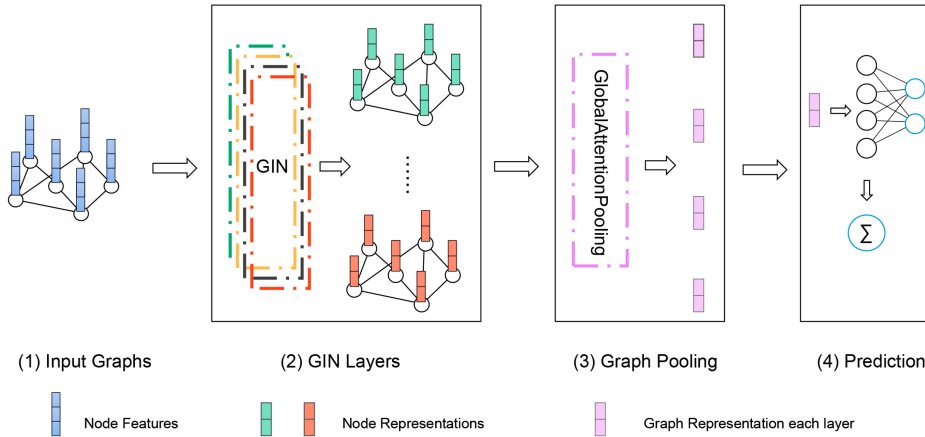


Figure 4.7: Architecture of a Graph Isomorphism Network (GIN). The model passes graph-structured input through multiple GIN layers with pooling to form graph-level representations, which are concatenated and fed into a classifier.

In this study, *GIN* was implemented to test the role of fine-grained structural features in identifying money laundering activity. The model builds on the theoretical foundation of the Weisfeiler–Lehman test for graph isomorphism, making it particularly well-suited for distinguishing subtle variations in graph structures that may indicate illicit fund flows.

The architecture comprised of three sequential GIN layers, each having *multi-layer perceptron* aggregators to enable rich feature transformations during message passing. A consistent

64-dimensional hidden representation was maintained across all layers, and this helped to balance the computational efficiency with expressive capacity.

Training stability was supported by standardisation of node features and careful parameter tuning. These refinements ensured that the model could effectively learn from diverse transaction patterns present in cryptocurrency networks.

4.8 Model Deployment Architecture

The inference pipeline was implemented in `main.py` and structured into the following sequence:

1. **Transaction Collection:** Real-time backward tracing from the target address using a BFS algorithm, with the trace data stored in a `.json` file
2. **Data Processing:** Feature extraction and graph conversion performed via the custom `GNNPreprocessor` class
3. **Multi-Model Inference:** Predictions generated in parallel from all four trained models to output the final inference results

The system produces an output structure containing model-specific predictions and confidence levels, as shown in Fig 4.8:

```
*** Inference (Random Forest) ***
Malicious Probability: 0.8400
Predicted Label: Malicious (1)

*** Inference (GraphSAGE) ***
Malicious Probability: 0.9752
Predicted Label: Malicious (1)

*** Inference (GAT) ***
Malicious Probability: 0.9661
Predicted Label: Malicious (1)

*** Inference (GIN) ***
Malicious Probability updated: 0.9928
Predicted Label: Malicious (1)
```

Figure 4.8: Model Predictions for a Sample Trace

This approach provides a streamlined pipeline for evaluating transaction data, where each model independently produces its own risk prediction. The system thus enables comparative analysis of how traditional machine learning and graph neural networks capture different aspects of money laundering behaviour within blockchain transaction patterns.

4.9 Implementation Challenges

API Rate Limiting

During data acquisition, the system relied on Etherscan’s free-tier API, which imposes a cap of 5 requests per second. While sufficient for prototyping, this restriction created bottlenecks in comprehensive backwards tracing, where dozens of queries may be required to fully reconstruct transaction paths. Such rate limits introduce latency and occasional service interruptions, particularly when operating at scale.

Mitigation: A practical solution is deploying a dedicated Ethereum node, which enables direct access to the blockchain ledger without external rate restrictions. This approach would substantially reduce latency and support more reliable transaction tracing in production settings.

Data Quality and Consistency

A second challenge is concerned with tracing malicious transaction flows. In practice, for building the dataset, manual tracing was required to follow laundering pathways until they reached endpoints such as cross-chain bridges, *CEX*, *DEX*, or mixers. At these points, visibility into further movements was lost, as these services typically aggregate or obfuscate flows. Consequently, malicious graphs tended to be structurally smaller compared to normal transaction graphs.

Mitigation: In real-world deployments, integrating additional data sources (e.g., exchange *KYC* records, cross-chain monitoring services) would help extend traceability and mitigate graph size imbalance between malicious and normal samples.

Chapter Five

Evaluation Results and Analysis

5.1 Overview

This chapter presents a comprehensive evaluation of the proposed real-time risk assessment system for detecting cryptocurrency money laundering. The evaluation is structured around three key research questions that address the core objectives of this research.

5.2 Evaluation Setup

5.2.1 Dataset Configuration

The experimental evaluation was conducted on the custom dataset of 201 transaction graphs, with 100 representing malicious money laundering patterns and 101 representing legitimate transaction behaviours. Table A.2 summarises the essential characteristics of the datasets employed in this study, encompassing the total number of wallets, transactions and the average number of transactions per wallet. ‘#’ means the number.

For all the models, the dataset was split using an 80:20 split, ensuring balanced representation of both classes in training (*160 graphs*) and testing (*41 graphs*) sets. This approach maintains the natural distribution of malicious and benign patterns while providing enough data for reliable model assessment.

5.2.2 Evaluation Metrics Selection

For this study, the following six validation metrics were used to comprehensively evaluate the performance of different models for Ethereum money laundering detection:

- **Accuracy:** Overall correctness of predictions across all samples

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} \quad (5.1)$$

- **Precision:** Proportion of correctly identified malicious transactions among all flagged transactions

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \quad (5.2)$$

- **Recall (Sensitivity):** Proportion of actual malicious transactions successfully detected

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \quad (5.3)$$

- **F1-Score:** Harmonic mean of precision and recall, providing balanced performance assessment

$$F1 = \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}} \quad (5.4)$$

- **ROC-AUC:** Area under the receiver operating characteristic curve, measuring discriminative ability

$$\text{ROC-AUC} = \int_0^1 \text{TPR}(\text{FPR}) d(\text{FPR}) \quad (5.5)$$

where

$$\begin{aligned} \text{TPR} &= \frac{TP}{TP + FN} && (\text{True Positive Rate}) \\ \text{FPR} &= \frac{FP}{FP + TN} && (\text{False Positive Rate}) \end{aligned}$$

- **PR-AUC (Average Precision):** Area under the precision-recall curve, particularly relevant for imbalanced datasets

$$\text{PR-AUC} = \int_0^1 \text{Precision}(\text{Recall}) d(\text{Recall}) \quad (5.6)$$

The selection of these metrics reflects the practical requirements of a production system where minimising false positives (*to avoid blocking legitimate transactions*) must be balanced against maximising true positive detection (*to catch illicit activities*).

5.3 RQ1: Model Effectiveness Analysis

To address RQ1, we evaluate the performance of all the models for the detection of illicit activities. The corresponding results are shown in Table 5.1.

Table 5.1: Model Performance Metrics

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC
Random Forest	60.00%	59.10%	65.00%	61.90%	0.7112	0.7447
<i>GraphSAGE</i>	75.00%	67.90%	65.00%	61.90%	0.7500	0.6696
<i>GAT</i>	82.50%	80.90%	85.00%	82.90%	0.9050	0.9091
<i>GIN</i>	87.50%	95.00%	82.61%	88.37%	0.9540	0.9726

5.3.1 Key Findings

The evaluation demonstrates a clear advantage of *GNNs* over the *RF* model for detecting illicit blockchain activities. All three *GNN* architectures surpassed the traditional approach across every metric, with improvements ranging from 15 to over 35 percentage points. This highlights the value of explicitly modelling relational structures within transaction graphs rather than relying on handcrafted statistical features alone.

A performance hierarchy emerges, with *GIN* consistently outperforming the other models, followed by *GAT*, *GraphSAGE*, and lastly *RF*. Notably, *GIN* achieved the highest overall accuracy (87.5%), precision (95%), F1-score (88.37%), and ROC-AUC/PR-AUC values, establishing it as the most reliable architecture for this task. *GAT* also performed strongly, with the highest recall (85%), indicating its strength in correctly identifying suspicious addresses, while *GraphSAGE* offered moderate improvements but lagged behind the more expressive attention and isomorphism-based models.

One of the most critical findings concerns precision. The jump from 59.1% (*RF*) to 95% (*GIN*) represents a reduction in false positives that is particularly significant for real-world deployment, where incorrectly flagging benign wallets can carry serious operational and reputational costs. This precision gain ensures that alerts raised by the system are far more trustworthy.

The results further suggest that different *GNN* architectures capture complementary aspects of transaction dynamics. While *GAT*'s strong recall emphasises its ability to detect a wider range of malicious patterns, *GIN*'s mathematical formulation provides superior discrimination, enabling it to filter noise more effectively.

Taken together, these findings provide a conclusive answer to RQ1: *GNNs* not only outperform traditional machine learning in blockchain-based money laundering detection, but the choice of architecture also matters. *GIN*, in particular, delivers the best balance of accuracy, robustness, and interpretability, marking it as the most promising candidate for production-level systems.

5.3.2 Feature Behaviour Analysis

In addition to aggregate model metrics, it is important to examine the behaviour of individual features that contribute to classification. One such feature is the *time delta* or fund velocity between consecutive transactions. Figure A.4 shows the distribution of scaled log-transformed time deltas for malicious and normal transactions.

The plot highlights a clear behavioural distinction that malicious transactions are often characterised by shorter time intervals, which shows the tendency of laundering patterns to move funds rapidly across accounts to obscure their origin. In contrast, normal transactions exhibit a broader and more evenly distributed range of time deltas, consistent with typical user activity.

However, there is also some overlap between the two distributions. Certain legitimate behaviours, such as frequent trading or automated arbitrage, can also produce short-time deltas similar to those observed in laundering activity. This overlap can be one of the reasons

why the models occasionally misclassify benign transactions as malicious, contributing to the false positives observed in the evaluation.

5.4 RQ2: Production Viability Assessment

For autonomous blocking in real-time *DeFi* systems, tolerance for error is extremely low. Typically, for such decentralised financial systems where everything operates based on smart contract codes, the blocking systems target precision must achieve at least 99.99% of all daily activity for it to be a viable option. As of August 2025, the Ethereum blockchain processes over 1.5 million transactions daily on average¹. According to CoinLedger’s 2024 Crypto Crime Report², roughly 0.14% of these are associated with illicit activity, about $\approx$ 2,100 truly malicious transactions per day that would need to be intercepted.

In this study, *GIN* was the best performing model and it achieved a precision of 95%. So that means it would misclassify 5%, which, when applied to Ethereum’s daily volume, corresponds to $\approx$ 75,000 false positives each day.

At this scale, the system would generate enough false positives to disrupt legitimate user activity on a daily basis, thus creating friction for end-users and posing reputational risks for various *DeFi* protocols and services.

Therefore, with respect to RQ2, we conclude that the proposed model’s performance falls well below the expected threshold for autonomous transaction blocking in production environments. While the model may still be valuable for alert generation and risk scoring, the precision requirements for fully autonomous blocking remain a significant challenge.

5.5 RQ3: Real-Time Performance Evaluation

Table 5.2 reports the average computational breakdown across the pipeline. It is clear that both preprocessing and model inference are extremely efficient, with all models operating well under 0.15 seconds per transaction on average (See Fig A.5). The major cost arises from the tracing step, which ranged anywhere from around 5 seconds to 550 seconds during testing. It was due to multiple API calls and dependency on external data sources.

Table 5.2: Average Pipeline Processing Times (in seconds)

Tracing	Preprocessing	RF	SAGE	GAT	GIN	Total
188.97	0.032	0.056	0.098	0.138	0.078	189.38

¹<https://etherscan.io/chart/tx>

²<https://coinledger.io/research/crypto-crime-report>

Acceptable Response Times

Given Ethereum’s average block confirmation time of ≈ 12 seconds, the sub-second inference speed ensures that risk scoring itself is not a bottleneck. However, replacing dependence on external data sources and APIs with a dedicated Ethereum node can substantially cut down these tracing delays.

Answer to RQ3: Currently, the models achieve sub-second inference times, under 0.15 seconds per trace, comfortably meeting real-time requirements. The main bottleneck is the tracing step, which is currently slowed by dependence on external APIs. Deploying a dedicated Ethereum node can significantly reduce tracing delays, enabling true real-time transaction screening.

Chapter Six

Conclusion and Future Work

6.1 Research Summary

This dissertation aimed to investigate the viability of using Machine Learning models such as *GNNs* to address the critical challenge of detecting money laundering in *DeFi* systems. By developing a labelled transaction dataset after analysing over 70 major crypto exploits and contrasting their laundering behaviours (*e.g.*, *peeling chains*, *dusting*, *mixer interactions*) with legitimate user activity, engineering a rich set of node and edge features, and conducting a comparative analysis of traditional and graph-based machine learning models, this work has yielded several key insights into the challenges and opportunities of automated *AML* solutions on the blockchain.

The research successfully addressed three key questions regarding the effectiveness of the model, production viability, and its real-time performance. The results conclusively demonstrated that *GNNs* substantially outperform traditional baseline models like *RF* in detecting illicit blockchain activities, with improvements ranging from 15 to over 35 percentage points across various metrics. *GIN* was the best performing model overall, and it was able to achieve this by assigning greater weight to critical high-risk interactions and effectively learning and detecting subtle structural patterns. This represents a significant advancement in reducing false positives that would otherwise disrupt legitimate user activity.

However, the evaluation revealed an important gap between current model performance and production deployment requirements. The *GIN* model’s 95% precision is impressive, but it falls short of the near-perfect accuracy needed for autonomous transaction blocking in high-volume environments like Ethereum. When compared to the network’s daily transaction volume, even a 5% false positives would be unacceptably disruptive to legitimate users. As a result, while the system may be viable for generating alerts and risk scoring to aid manual analysis, it is not yet ready for fully automated blocking of transactions.

The real-time performance analysis demonstrated that model inference operates well within acceptable timeframes (*under 0.15 seconds*), with the primary bottleneck occurring in transaction tracing due to the current system’s API rate limitations rather than computational constraints. This suggests that the core ML framework is production-ready from a performance perspective, with infrastructure optimisation being the primary requirement for deployment.

6.2 Future Work

Building on the findings and limitations of this study, several promising directions for future research emerge:

- **Dataset Expansion and Enrichment:** The dataset, though representative, remains limited in scope. Expanding the collection to include larger transaction graphs, more diverse laundering patterns, and multi-chain data would improve model robustness and generalisability. Integration with labelled datasets from compliance providers could further enhance realism.
- **Data Pipeline Optimisation :** Future iterations should replace the current reliance on third-party APIs by deploying a dedicated, full Ethereum node. This would grant unrestricted access to blockchain data, dramatically reducing latency and making the end-to-end pipeline truly real-time.
- **Advanced Model Architecture & Human-in-the-Loop Learning:** Achieving the 99.99% precision required for autonomous transaction blocking is an ambitious but necessary target. Future research should investigate advanced GNN architectures, novel feature engineering techniques, or ensemble methods to further reduce false positives without compromising the detection of sophisticated illicit flows. An important complementary direction is incorporating human-in-the-loop learning, where analyst feedback on flagged cases is systematically used to retrain or fine-tune the models. This continuous feedback loop would allow the system to adapt to evolving laundering patterns more quickly and improve both accuracy and trustworthiness in production settings.
- **Adversarial Robustness Testing:** As criminals actively adapt their methods, future work should focus on developing strategies to enhance the system's robustness against adversarial attacks, where illicit actors try to mimic benign behaviour to evade detection. This could involve adversarial training techniques or more sophisticated anomaly detection algorithms.
- **Extending Generalisability to Other Blockchains:** While the current methodology is designed to be generalisable, its implementation and evaluation were exclusively within the Ethereum ecosystem. Future work could expand and adapt the methodology to other major blockchain networks, including Bitcoin, Binance Smart Chain, Polygon, Solana and emerging layer-1 & layer-2 platforms.

Appendix One

Appendix

A.1 Project Repository

The complete source code developed for this dissertation is available at: <https://git.cs.bham.ac.uk/yxn417/Code>

This repository contains the data collection scripts, preprocessing pipeline, model training code, and inference modules used throughout the study.

A.2 Risky Address Data Sources

The list of risky addresses used for constructing malicious traces was compiled from multiple sources to ensure comprehensive coverage of laundering patterns. The datasets were sourced from:

- *ScamSniffer – Github*: <https://github.com/scamsniffer/scam-database/blob/main/blacklist/address.json>
- *OFAC Sanctioned Addresses - Etherscan*: <https://etherscan.io/accounts/label/ofac-sanctioned>
- *Dune Analytics*: <https://dune.com/data>

These lists were further supplemented by manual enumeration based on incident reports and blockchain security analyses.

A.3 Ethereum Transaction Fields

Table A.1 provides a reference for the key fields extracted from Ethereum transaction traces. In addition to the standard blockchain metadata, custom fields such as `is_risky_tx` and `risk_type_tx` were introduced during preprocessing to label suspicious activity.

A.4 Dataset Statistics

Table A.2 summarises the dataset used for evaluation, highlighting the distribution of wallets and transactions across malicious and benign categories.

Table A.1: Ethereum Transaction Trace Fields and Their Descriptions

Field	Description
blockNumber, block	Block number in which the transaction was included.
timeStamp	Unix timestamp (seconds) of when the block was mined.
hash, fundingTxn	Unique identifier of the transaction.
nonce	Number of transactions sent from the sender's address. Prevents replay attacks.
blockHash	Hash of the block containing the transaction.
transactionIndex	Position of the transaction within the block.
from, fundingAddress	Address of the account initiating the transaction.
to	Recipient address (wallet or contract).
value	Amount of Ether (in Wei) transferred.
gas, gasPrice	Gas limit and price per unit set by the sender.
cumulativeGasUsed, gasUsed	Cumulative and actual gas consumption.
isError, txreceipt_status	Transaction success/failure indicators.
input, methodId, functionName	Smart contract call data and function selector.
contractAddress	Contract created by this transaction (if applicable).
confirmations	Number of blocks confirmed after inclusion.
transaction_type	Transaction type (normal, token, or internal).
is_risky_tx, risk_type_tx	Custom labels for risky transactions and their category.
tokenName, tokenSymbol, tokenDecimal	ERC-20 token metadata (if applicable).

A.5 Supplementary Figures

Figures A.1–A.5 present supplementary illustrations that support the main evaluation. These include a depiction of time delta for fund movements, a breakdown of risky address categories, examples of risky transaction traces, and detailed performance metrics across models.

Table A.2: Transaction Statistics Across Malicious and Normal Datasets

Dataset	# Files	# Wallets	# Transactions	Avg. Tx/Wallet
Malicious	100	12,331	9,728	0.79
Normal	101	35,876	52,990	1.48
Total	201	45,143	62,718	1.39

```

Dataset > {} riskyAddresses.json > ...
  1  {
  2  >   "CEX": [ ...
4451   ],
4452 >   "DEX": [ ...
4602   ],
4603 >   "mixers": [ ...
4610   ],
4611 >   "bridges": [ ...
4867   ],
4868 >   "sanctionedAddresses": [ ...
4915   ],
4916 >   "knownAttackers": [ ...
9862   ]
9863 }
```

Figure A.1: Distribution of risky address categories compiled for this study.

```

1 {
2   "0x70ea8c85608bee0b18fb5c547b95f28335d63eea": {
3     "depth": 0,
4     "funded_by_address": {
5       "block": 22266097,
6       "timeStamp": "1744620059",
7       "fundingAddress": "0xac5b4e1d9c2f6c103cb381d5b8f02dda3ad3342f",
8       "fundingTxn": "0x844d065f9fe0aeca2bafd709101ec2988ed988602ef449acb2522a47b11d2b0",
9       "value": "98986689593697003000"
10    },
11    "incoming_transactions_traced": [
12      {
13        "blockNumber": "22266097",
14        "blockHash": "0x7fb621858446032a5323f9bf454a887566ab2d06b324d2c7d857ff786821f215",
15        "timeStamp": "1744620059",
16        "hash": "0x844d065f9fe0aeca2bafd709101ec2988ed988602ef449acb2522a47b11d2b0",
17        "nonce": "0",
18        "transactionIndex": "92",
19        "from": "0xac5b4e1d9c2f6c103cb381d5b8f02dda3ad3342f",
20        "to": "0x70ea8c85608bee0b18fb5c547b95f28335d63eea",
21        "value": "98986689593697003000",
22        "gas": "21000",
23        "gasPrice": "1399537618",
24        "input": "0x",
25        "methodId": "0x",
26        "functionName": "",
27        "contractAddress": "",
28        "cumulativeGasUsed": "4142029",
29        "txreceipt_status": "1",
30        "gasUsed": "21000",
31        "confirmations": "830072",
32        "isError": "0",
33        "transaction_type": "normal",
34        "is_risky_tx": true,
35        "risk_type_tx": "knownAttackers"
36      }
37    ],
38    "outgoing_transactions_monitored": [
39

```

Figure A.2: Example of a transaction flagged as risky with interaction markers for a known attacker.

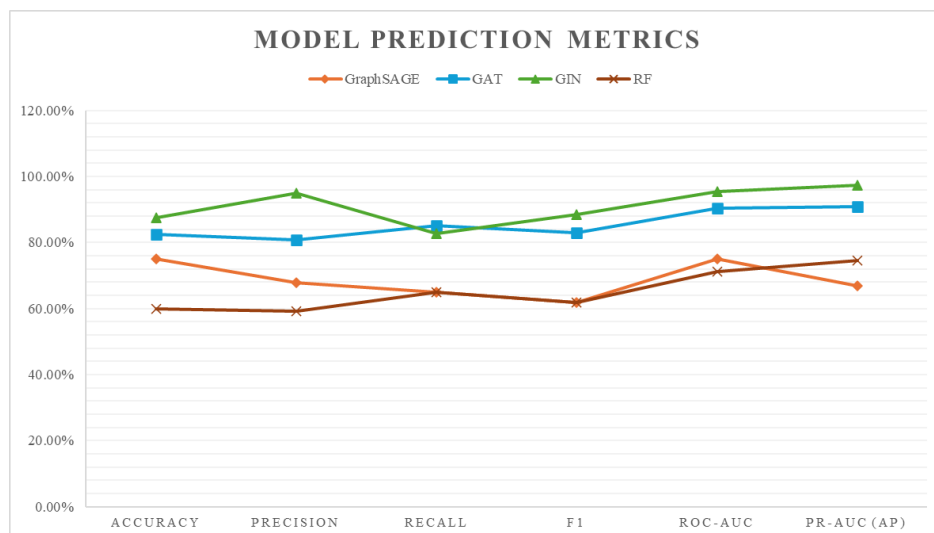


Figure A.3: Detailed performance metrics for different model architectures.

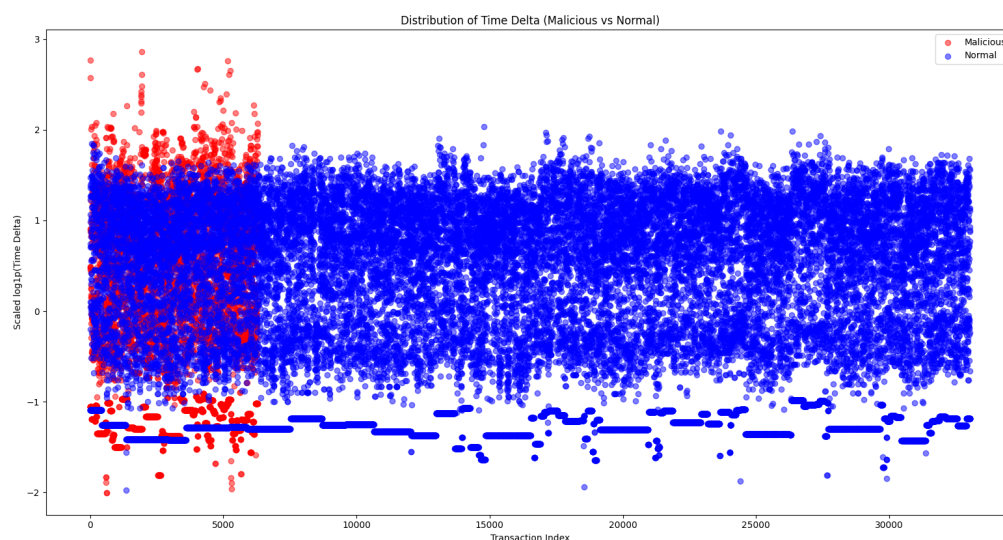


Figure A.4: Time delta distribution comparing malicious (red) vs normal (blue) transactions, with lower deltas indicating faster fund movement

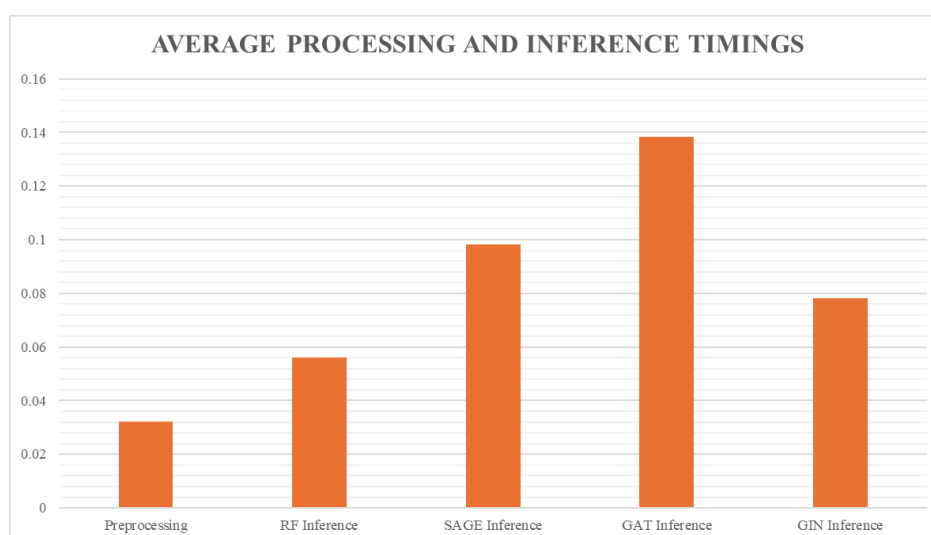


Figure A.5: Average preprocessing and model inference times across addresses.

References

- [1] *Live cryptocurrency charts & market data / CoinMarketCap*. URL: <https://coinmarketcap.com/charts/>.
- [2] *Rekt - the one that got away*. Aug. 7, 2025. URL: <https://rekt.news/the-one-that-got-away>.
- [3] *Rekt - ByBit - Rekt*. Feb. 22, 2025. URL: <https://rekt.news/bybit-rekt>.
- [4] Esq Browne et al. “Money Laundering and Cryptocurrency: Are We Moving in the Right Direction?” In: *Available at SSRN 5104082* (2023).
- [5] Céline Carucci. *Anti-money Laundering in the Age of Cryptocurrencies*. Vol. 10. buch & netz, 2024.
- [6] Ebberth L Paula et al. “Deep learning anomaly detection as support fraud investigation in brazilian exports and anti-money laundering”. In: *2016 15th ieee international conference on machine learning and applications (icmla)*. IEEE. 2016, pp. 954–960.
- [7] Hilmar Páll Stefánsson et al. “Detecting potential money laundering addresses in the Bitcoin blockchain using unsupervised machine learning.” In: *HICSS*. 2022, pp. 1–10.
- [8] Anton Olsson and Daniel Andersson. “The dark flows of cryptocurrency: an overview of money flow behaviors in bitcoin transactions related to online criminal activities and bitcoin mixers”. In: (2024).
- [9] Yining Hu et al. “Characterizing and detecting money laundering activities on the bitcoin network”. In: *arXiv preprint arXiv:1912.12060* (2019).
- [10] Marc Jourdan et al. “Characterizing entities in the bitcoin blockchain”. In: *2018 IEEE international conference on data mining workshops (ICDMW)*. IEEE. 2018, pp. 55–62.
- [11] Joana Lorenz et al. “Machine learning methods to detect money laundering in the bitcoin blockchain in the presence of label scarcity”. In: *Proceedings of the first ACM international conference on AI in finance*. 2020, pp. 1–8.
- [12] Elliptic. *Elliptic data set for Anti-Money laundering in Bitcoin*. URL: <https://www.elliptic.co/media-center/elliptic-releases-bitcoin-transactions-data>.
- [13] Xinwei Deng et al. “Active learning through sequential design, with applications to detection of money laundering”. In: *Journal of the American Statistical Association* 104.487 (2009), pp. 969–981.

-
- [14] Shiyu Ouyang et al. “Bitcoin money laundering detection via subgraph contrastive learning”. In: *Entropy* 26.3 (2024), p. 211.
 - [15] Mark Weber et al. “Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics”. In: *arXiv preprint arXiv:1908.02591* (2019).
 - [16] Hrushyang Adloori, Vaishnavi Dasanapu, and Abhijith Chandra Mergu. “Graph Network Models To Detect Illicit Transactions In Block Chain”. In: *arXiv preprint arXiv:2410.07150* (2024).
 - [17] Xiangfeng Li et al. “Flowscope: Spotting money laundering based on graphs”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 34. 04. 2020, pp. 4731–4738.
 - [18] Xiaobing Sun et al. “MonLAD: Money laundering agents detection in transaction streams”. In: *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*. 2022, pp. 976–986.
 - [19] Dan Lin et al. “Towards understanding crypto money laundering in web3 through the lenses of ethereum heists”. In: *arXiv preprint arXiv:2305.14748* (2023).
 - [20] Devraj Basu and Godsway Korku Tetteh. “Using automation and AI to combat money laundering”. In: (2024).
 - [21] *Rekt - Ronin Network - REKT*. Mar. 29, 2022. URL: <https://rekt.news/ronin-rekt>.
 - [22] *Rekt - Fei Rari - REKT 2*. May 1, 2022. URL: <https://rekt.news/fei-rari-rekt>.
 - [23] *Elliptic Data set*. July 31, 2019. URL: <https://www.kaggle.com/datasets/ellipticco/elliptic-data-set>.
 - [24] Leo Breiman. “Random forests”. In: *Machine learning* 45.1 (2001), pp. 5–32.
 - [25] William L. Hamilton, Rex Ying, and Jure Leskovec. “Inductive representation learning on large graphs”. In: *arXiv (Cornell University)* (Jan. 1, 2017). DOI: 10.48550/arxiv.1706.02216. URL: <https://arxiv.org/abs/1706.02216>.
 - [26] Petar Veličković et al. “Graph attention networks”. In: *arXiv (Cornell University)* (Jan. 1, 2017). DOI: 10.48550/arxiv.1710.10903. URL: <https://arxiv.org/abs/1710.10903>.
 - [27] Keyulu Xu et al. “How Powerful are Graph Neural Networks?” In: *arXiv (Cornell University)* (Jan. 1, 2018). DOI: 10.48550/arxiv.1810.00826. URL: <https://arxiv.org/abs/1810.00826>.
 - [28] Boris Weisfeiler and Andrei Leman. “The reduction of a graph to canonical form and the algebra which appears therein”. In: *nti, Series* 2.9 (1968), pp. 12–16.
 - [29] Francesco Ernesto Alessi Longa. “Cryptocurrency and Money Laundering”. In: *American Journal of Industrial and Business Management* 15.2 (2025), pp. 362–371.

Websites consulted

- Etherscan – <https://etherscan.io/>
- Rekt News – <https://rekt.news/>
- Arkham Intelligence – <https://intel.arkm.com/>